■ Design a CI/CD Pipeline System

System Design Interview | Asynchronous Systems

algomaster.io | Deep Dive Notes

■ Key Requirements

Functional	Trigger on git push; Run build, test, lint, security scan stages; Deploy to staging then production; Rollback support; Artifact storage; Notifications
Non-Functional	Build starts <30s of push; Support 1000s of concurrent builds; Isolated build environments; Audit trail; Secrets management; 99.9% availability
Scale	GitHub Actions: millions of workflow runs/day; Netflix: 1000s of deploys/day; build parallelism is key to speed

■ High-Level Architecture

• Git Event → Webhook → Pipeline Service → Job Queue (Kafka/SQS) → Build Agents (ephemeral containers) → Artifact Store (S3) → Deploy Service

• Pipeline Service: parses pipeline config (.yml); creates DAG of stages/jobs; tracks overall pipeline state

• Build Agents: ephemeral Docker containers or VMs; spun up per job, destroyed after; ensures clean isolated environment

• Artifact Registry: stores built binaries, Docker images, test reports; tagged with commit SHA for traceability

■ Pipeline Execution Engine

• **DAG Scheduler:** Pipeline stages form a DAG; parallel jobs within a stage run concurrently; next stage starts only when all prior jobs pass

• **Job Dispatch:** Coordinator picks eligible jobs (deps satisfied); publishes to queue; agent picks up, clones repo at commit SHA, runs steps

• **Workspace Cache:** Cache node_modules / .m2 / pip packages in S3 or distributed cache; restore on agent start; reduces build time by 50-80%

• **Layer Caching:** Docker build cache: reuse layers if Dockerfile + dependencies unchanged; push cache to ECR/registry for agent reuse

• **Parallelism:** Matrix builds: same job runs across multiple OS/versions in parallel; fan-out then fan-in (all must pass)

■ Deployment Strategies

• **Blue-Green:** Two identical environments; route traffic to blue; deploy to green; switch LB to green; instant rollback by switching back

• **Canary:** Deploy to 1-5% of servers first; monitor error rate and latency; gradually increase to 100% or rollback if metrics degrade

• **Rolling:** Replace instances one at a time; always N-1 instances serving; slow but zero-downtime; rollback by reversing direction

• **Feature Flags:** Deploy code to 100% of servers but gate feature behind flag; enable for % of users; decouple deploy from release

■ Security & Secrets Management

• Secrets never in code or config files; stored in Vault (HashiCorp) or AWS Secrets Manager; injected as env vars at runtime

• OIDC-based auth: build agent requests short-lived token from identity provider; no long-lived credentials stored

• SAST (Static Analysis): run on every PR; block merge if critical vulnerabilities found; tools: SonarQube, Semgrep

• Container image scanning: scan Docker images for CVEs before push to registry; block deploy if critical CVE found

• Audit log: every pipeline trigger, approval, deploy, rollback logged with actor, timestamp, commit SHA

■■ Build Agent Types

Ephemeral Containers (Docker)	Persistent VMs
✓ Clean environment every run	✓ Faster for large workloads
✓ Fast spin-up (seconds)	✓ Persistent cache on disk
✓ Easy to scale	✓ Harder to scale elastically
✓ No state leakage between builds	✓ Risk of environment drift

■ Observability & Flaky Test Detection

• Build metrics: queue wait time, build duration, success/failure rate, flaky test rate — track per repo and per job

• Flaky test detection: same test passes/fails non-deterministically; flag tests that flip > N times in last 30 runs; quarantine flaky tests

• Test result trending: if PR increases test duration by >20% → warn author; detect test regressions early

• Notification: Slack/email on failure with link to logs; only notify on first failure (not every retry); tag commit author